РЕУ имени Плеханова

ПРАКТИЧЕСКАЯ РАБОТА №2

по дисциплине

«Поддержка и тестирование программных модулей»

Тема: Создание калькулятора на WPF
с модульным тестированием

Выполнил:
студент группы ИПс-11
Кузнецов Андрей

ОГЛАВЛЕНИЕ

1. Введение	3
2. Ход работы	4
2.1. Создание структуры решения	4
2.2. Реализация библиотеки классов	5
2.3. Создание WPF-интерфейса	6
2.4. Модульное тестирование	7
2.5. Результат работы программы	8
3. Заключение	9
4. Список использованных источников	10

1. ВВЕДЕНИЕ

Цель работы: приобрести практические навыки создания приложений на платформе .NET с использованием технологии WPF, а также освоить принципы модульного тестирования программного кода.

Задачи:

- Создать приложение-калькулятор с арифметическими операциями (+, -, *, /, ^)
- Реализовать логику вычислений в виде отдельной библиотеки классов
- Создать пользовательский интерфейс с использованием WPF
- Написать модульные тесты для проверки корректности работы логики
- Разместить исходный код на платформе GitHub

Используемые технологии: C#, .NET 10.0, WPF, MSTest, Visual Studio 2026

2. ХОД РАБОТЫ

2.1. Создание структуры решения

В среде Visual Studio было создано решение, содержащее три проекта:

- **Kuznetsov.Calculator.Logic** — библиотека классов с логикой вычислений
- **Kuznetsov.Calculator.WPF** — WPF-приложение с пользовательским интерфейсом
- **Kuznetsov.Calculator.Tests** — проект модульных тестов

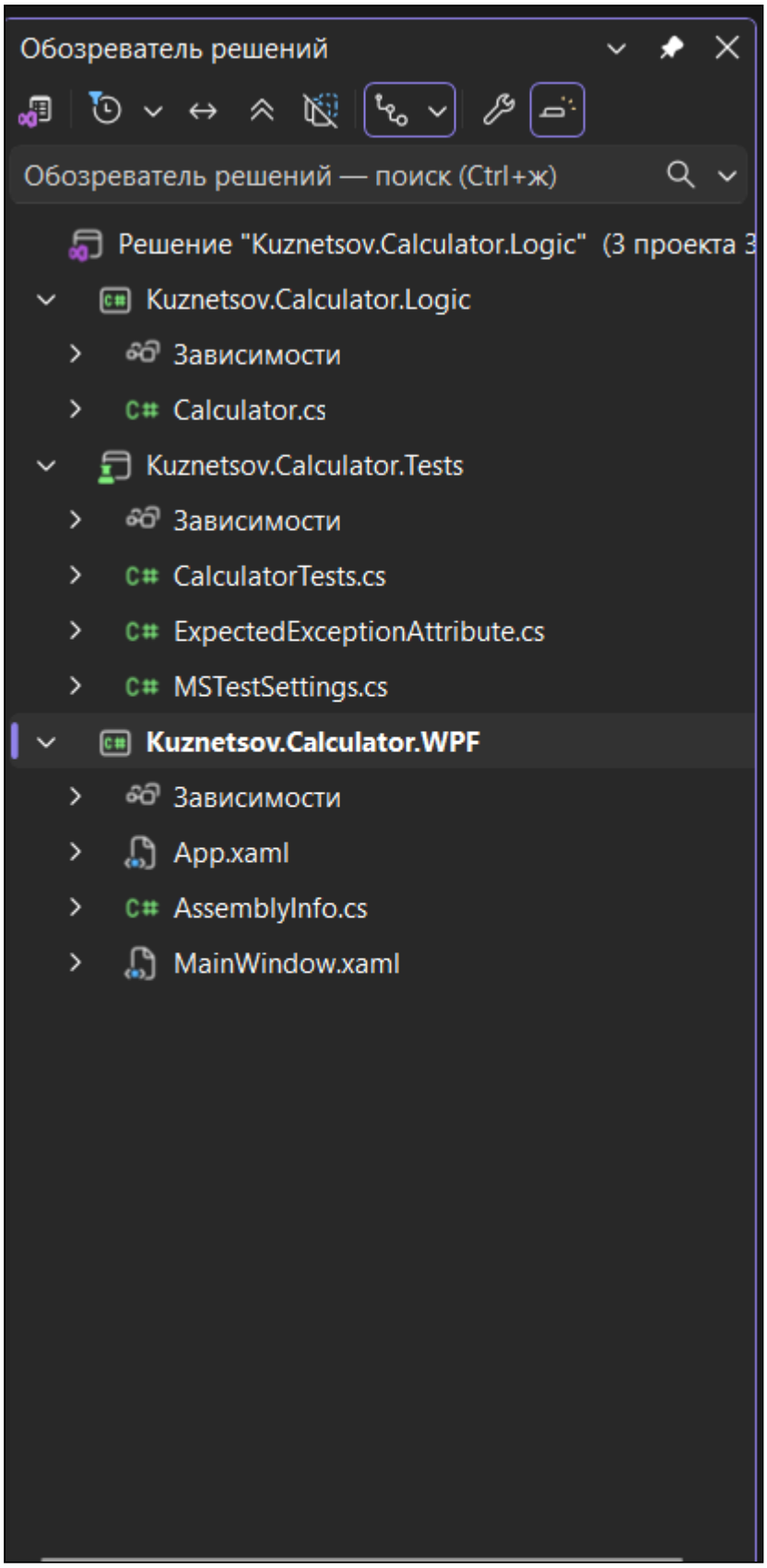

Рисунок 1 — Структура решения в Visual Studio

2.2. Реализация библиотеки классов

В проекте **Kuznetsov.Calculator.Logic** был создан класс `Calculator` со следующими методами:

```
namespace Kuznetsov.Calculator.Logic { public class Calculator { public double Add(double a, double b) => a + b; public double Subtract(double a, double b) => a - b; public double Multiply(double a, double b) => a * b; public double Divide(double a, double b) { ... } public double Power(double a, double b) => Math.Pow(a, b); } }
```

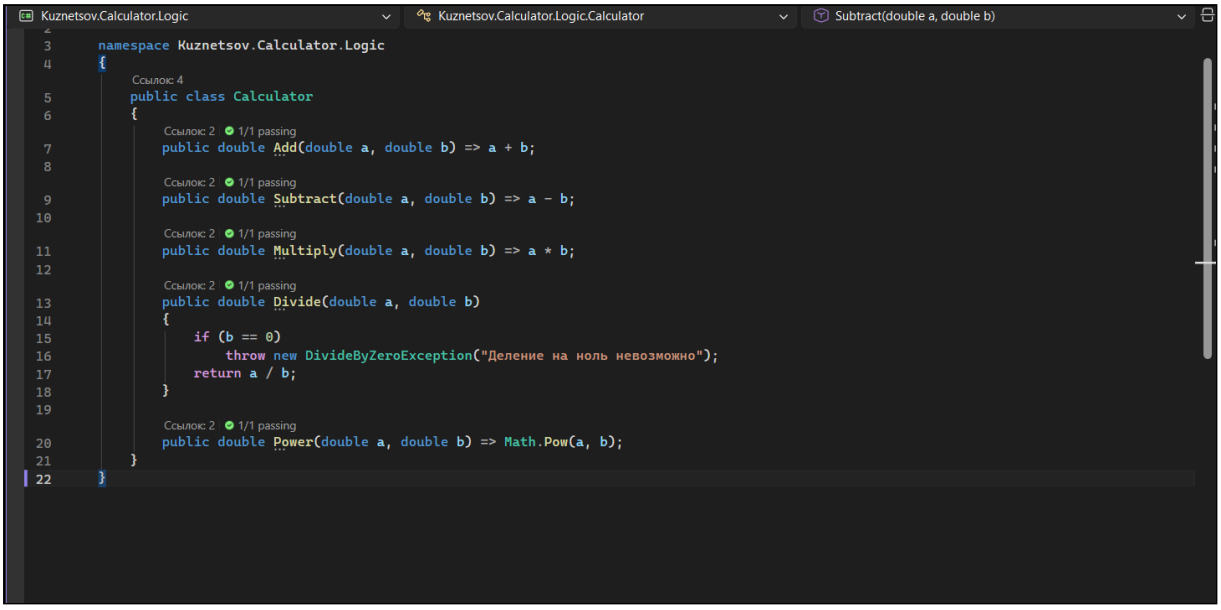

Рисунок 2 — Код класса Calculator

2.3. Создание WPF-интерфейса

Интерфейс приложения разработан с использованием XAML. Окно содержит:

- Два поля ввода для чисел
- Пять кнопок для операций (+, -, *, /, ^)
- Поле для отображения результата
- Поле для отображения ошибок

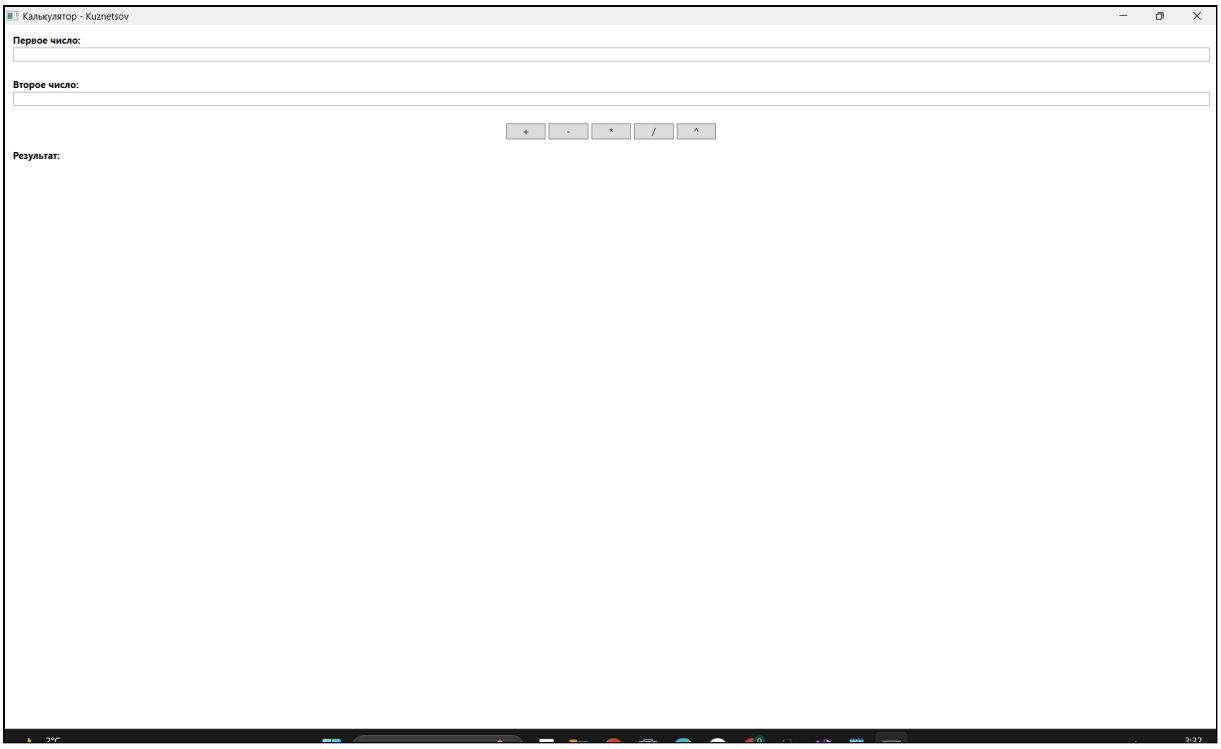

Рисунок 3 — Интерфейс приложения

2.4. Модульное тестирование

Для проверки корректности работы библиотеки было написано 5 модульных тестов:

№	Название теста	Проверяемая операция
1	Add_PositiveNumbers_ReturnsCorrectResult	Сложение
2	Subtract_NegativeResult_ReturnsCorrectResult	Вычитание
3	Multiply_Zero_ReturnsZero	Умножение
4	Divide_ValidInput_ReturnsCorrectResult	Деление
5	Power_Square_ReturnsCorrectResult	Возведение в степень

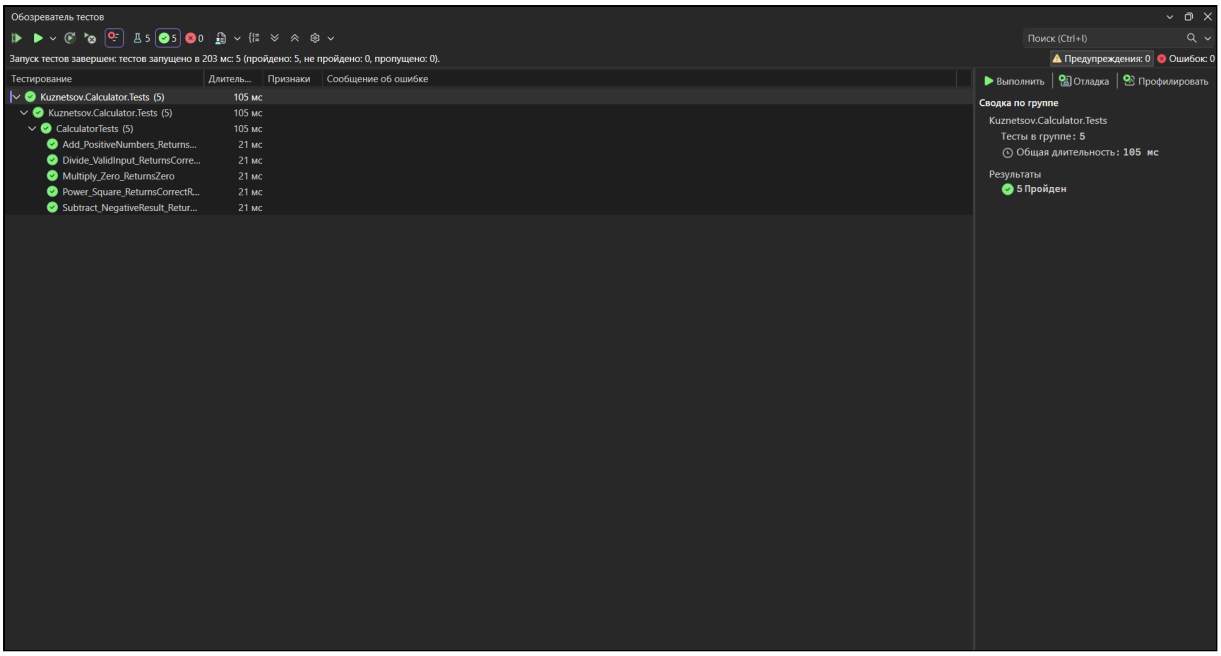

Рисунок 4 — Результаты модульного тестирования

2.5. Результат работы программы

Приложение успешно собрано и протестировано. Все арифметические операции выполняются корректно.

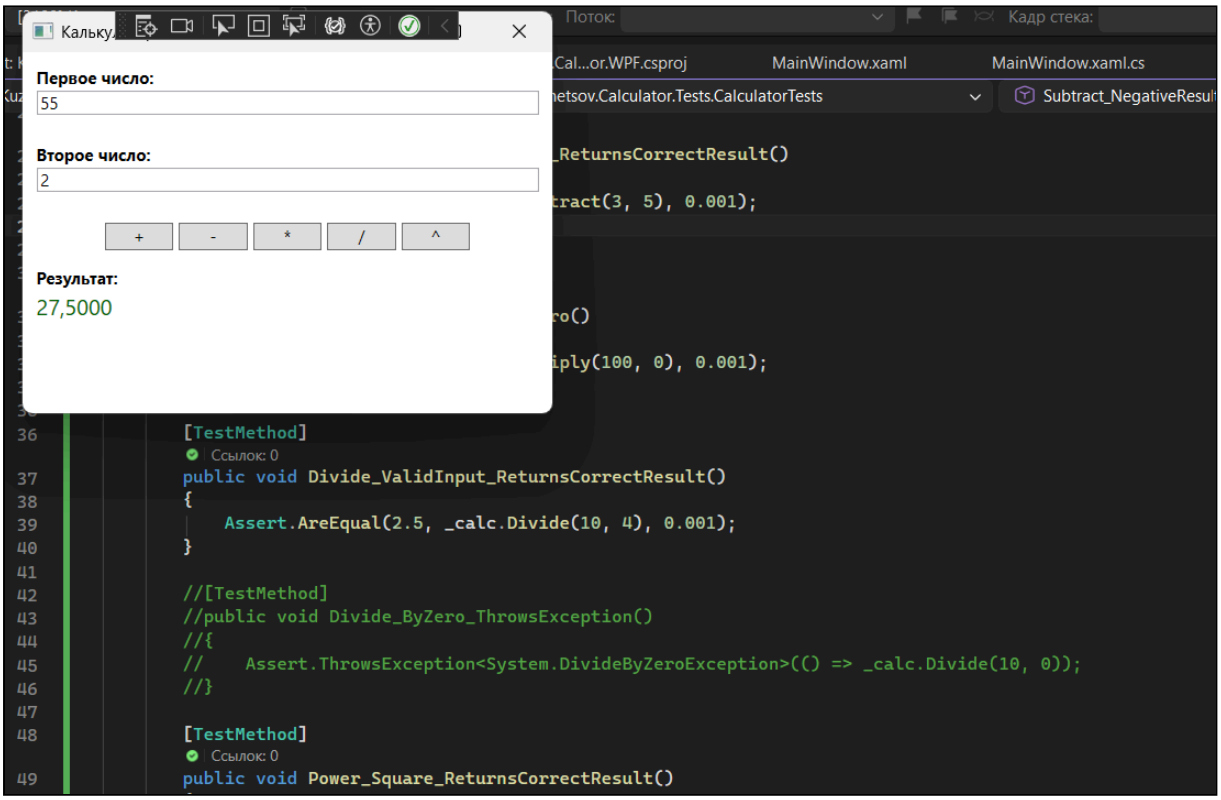

Рисунок 5 — Работающее приложение

3. ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы были освоены следующие навыки:

- Создание многопроектных решений в Visual Studio
- Разделение логики приложения и пользовательского интерфейса
- Разработка интерфейсов на технологии WPF
- Написание и запуск модульных тестов с использованием MSTest
- Работа с системой контроля версий Git и платформой GitHub

Все требования практической работы выполнены в полном объёме.

4. СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Metanit — Учебник по C# и .NET [Электронный ресурс]. — URL: https://metanit.com/sharp/tutorial/
2. Metanit — WPF приложения [Электронный ресурс]. — URL: https://metanit.com/sharp/wpf/
3. Microsoft Learn — Модульное тестирование [Электронный ресурс]. — URL: https://learn.microsoft.com/ru-ru/visualstudio/test/
4. CodeNet — Юнит-тестирование [Электронный ресурс]. — URL: https://codernet.ru/articles/drugoe/modulnoe_testirovanie_yunit-testirovanie/